

Informe Breve

Jerarquías

```
biblioteca/  
— model/  
|   — Libro.java  
|   — Usuario.java  
|   — Prestamo.java  
|   — Catalogo.java  
|   — EstadoPrestamo.java  
— view/  
|   — LibroView.java  
|   — LibroPanel.java  
— presenter/  
|   — LibroPresenter.java  
— main/  
    — Main.java
```

Polimorfismo en el proyecto

Jerarquía de Material

El polimorfismo está en toString(). Material tiene su versión base y Libro la sobrescribe:

```
java
```

```
// En Material
```

```
@Override
```

```
public String toString() {  
    return String.format("[%s] %s — %s (%d) | Disponibles: %d/%d",  
        codigo, titulo, autor, anio, copiasDisponibles, totalCopias);  
}
```

```
// En Libro — agrega su propio prefijo
```

```
@Override
```

```
public String toString() {  
    return "Libro | " + super.toString();  
}
```

Esto significa que si hay una referencia de tipo Material apuntando a un Libro, Java en tiempo de ejecución llama la versión de Libro:

```
java
```

```
Material m = new Libro(...);
```

```
System.out.println(m.toString()); // llama versión de Libro, no de Material
```

Jerarquía de Usuario

El polimorfismo está en mostrarDatos(). Usuario define los datos comunes y cada subclase agrega los suyos:

```
java
```

```
// Docente agrega departamento
```

```
@Override
```

```
public void mostrarDatos() {  
    super.mostrarDatos();  
    System.out.println("Departamento: " + departamento);  
}
```

```
// Estudiante agrega carrera y semestre
```

```
@Override
```

```
public void mostrarDatos() {  
    super.mostrarDatos();  
    System.out.println("Carrera: " + carrera);  
    System.out.println("Semestre: " + semestre);  
}
```

```
}
```

Y funciona así en tiempo de ejecución:

```
java
```

```
Usuario u1 = new Docente(...);
```

```
Usuario u2 = new Estudiante(...);
```

```
u1.mostrarDatos(); // datos base + departamento
```

```
u2.mostrarDatos(); // datos base + carrera + semestre
```

Sobrecarga en el proyecto

La sobrecarga está en los constructores de ambas jerarquías. Cada nivel agrega parámetros propios al constructor del padre:

Jerarquía de Material:

```
java
```

```
// Material — 5 parámetros
```

```
public Material(String codigo, String titulo, String autor, int anio, int totalCopias)
```

```
// Libro — 7 parámetros (agrega isbn y editorial)
```

```
public Libro(String codigo, String titulo, String autor, int anio,  
             int totalCopias, String isbn, String editorial)
```

Jerarquía de Usuario:

```
java
```

```
// Usuario — 5 parámetros
```

```
public Usuario(String nombre, String apellido, String carnet, String telefono, String correo)
```

```
// Docente — 6 parámetros (agrega departamento)
```

```
public Docente(String nombre, String apellido, String carnet,
```

String telefono, String correo, String departamento)

// Estudiante — 7 parámetros (agrega carrera y semestre)

```
public Estudiante(String nombre, String apellido, String carnet,  
                  String telefono, String correo, String carrera, int semestre)
```

Problemas y Soluciones

Conectar las ventanas y clases entre sí

Un problema que durante la segunda y tercera semana fue que cuando comenzamos a hacer las funcionalidades, cada uno trabajo una ventana diferente y esto fue bien hasta que tuvimos que conectarlas entre sí, ya que hasta el momento no habíamos hecho mucho con eso, o no habíamos visto mucho la clase main, pero lo solucionamos cambiando las arraylist a la clase main y que así sea accesible para las demás ventanas.

Actualización de copias disponibles en devoluciones

Un problema crítico fue que cuando se devolvía un material en la ventana de Préstamos, el sistema registraba la devolución, pero no actualizaba la cantidad de copias disponibles del material en la tabla de Materiales. Esto causaba que los materiales aparecieran con menos copias disponibles de las que realmente tenían. La solución fue llamar al método devolver () de la clase Material cuando se registra una devolución, que incrementa automáticamente el contador de copias disponibles, y luego refrescar la tabla de Materiales para mostrar los valores actualizados.

Listas desplegables para datos categóricos

Al registrar usuarios, se pedía que el usuario escribiera manualmente la carrera, semestre y departamento, lo que generaba inconsistencias por errores de escritura o variaciones en el formato. La solución fue reemplazar los JTextField por JComboBox con opciones predefinidas. Para Estudiantes se agregó un combobox de carreras y otro de semestres; para Docentes se agregó un combobox de departamentos. Esto asegura que todos los datos sean consistentes y válidos.

Mostrar préstamos activos de usuarios

En la ventana de Usuarios, el sistema mostraba siempre '0' en la columna de 'Préstamos activos', aunque el usuario tuviera préstamos registrados. El problema era que esta columna no se actualizaba cuando se registraban nuevos préstamos desde la ventana de Préstamos. La solución fue crear un método que cuente los préstamos activos de un usuario buscando en la lista de préstamos aquellos que coincidan con el carnet del usuario y que no hayan sido devueltos, actualizando la tabla cada vez que se registra un préstamo o devolución.

Generación de IDs únicos para préstamos

Durante la implementación de la ventana de Préstamos, notamos que todos los préstamos generaban el mismo ID porque el contador no se incrementaba correctamente. El problema residía en que el contador estaba siendo reiniciado cada vez que se abría la ventana. La solución fue hacer que el contador sea un atributo de instancia que persiste durante toda la sesión de la ventana, y además se incrementa automáticamente después de cada préstamo registrado, garantizando que cada nuevo préstamo tenga un ID único y diferente.

Validación de campos vacíos en Materiales y Usuarios

Otro problema que surgió durante el desarrollo fue que, al registrar materiales y usuarios, el sistema permitía guardar registros con campos vacíos. Esto causaba inconsistencias en la base de datos y errores al intentar buscar o mostrar información. La solución fue implementar validaciones en los métodos `registrarMaterial()` y `registrarUsuario()` que verifican que todos los campos obligatorios estén rellenos antes de agregar el registro a la lista. Si algún campo está vacío, se muestra un `JOptionPane` con un mensaje de error informando al usuario cuál campo debe completar.